

PROJECT 3: Relational Database Service (RDS) Implementation

1. Project Objective

The goal of this project is to provision a managed MySQL database instance using AWS RDS (Relational Database Service) and establish a secure connection from an Amazon EC2 instance to demonstrate database accessibility and management.

2. Architecture Overview

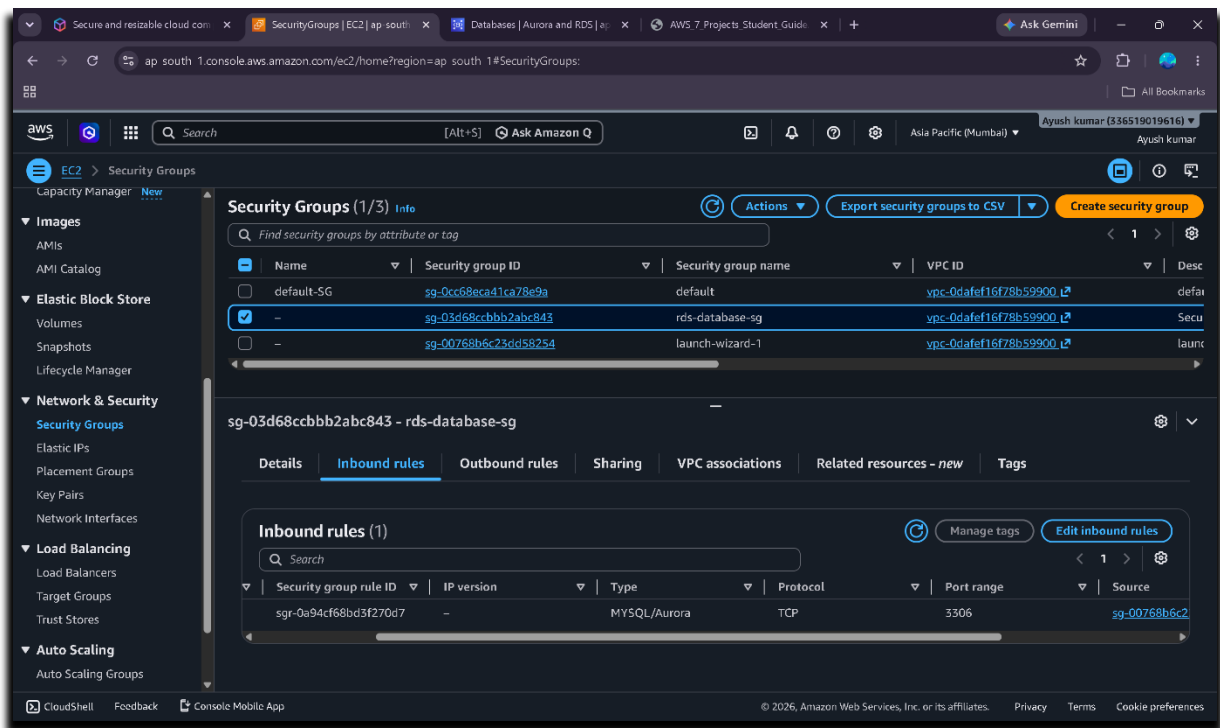
- **Database Engine:** MySQL
 - **Tier:** AWS Free Tier
 - **Connectivity:** Private Subnets with access restricted via Security Groups.
 - **Client:** Amazon EC2 Linux Instance.
-

3. Implementation Steps

Step 1: Configuring the Security Group for RDS

A dedicated security group was created to ensure that the database is only accessible from our specific web server (EC2 instance).

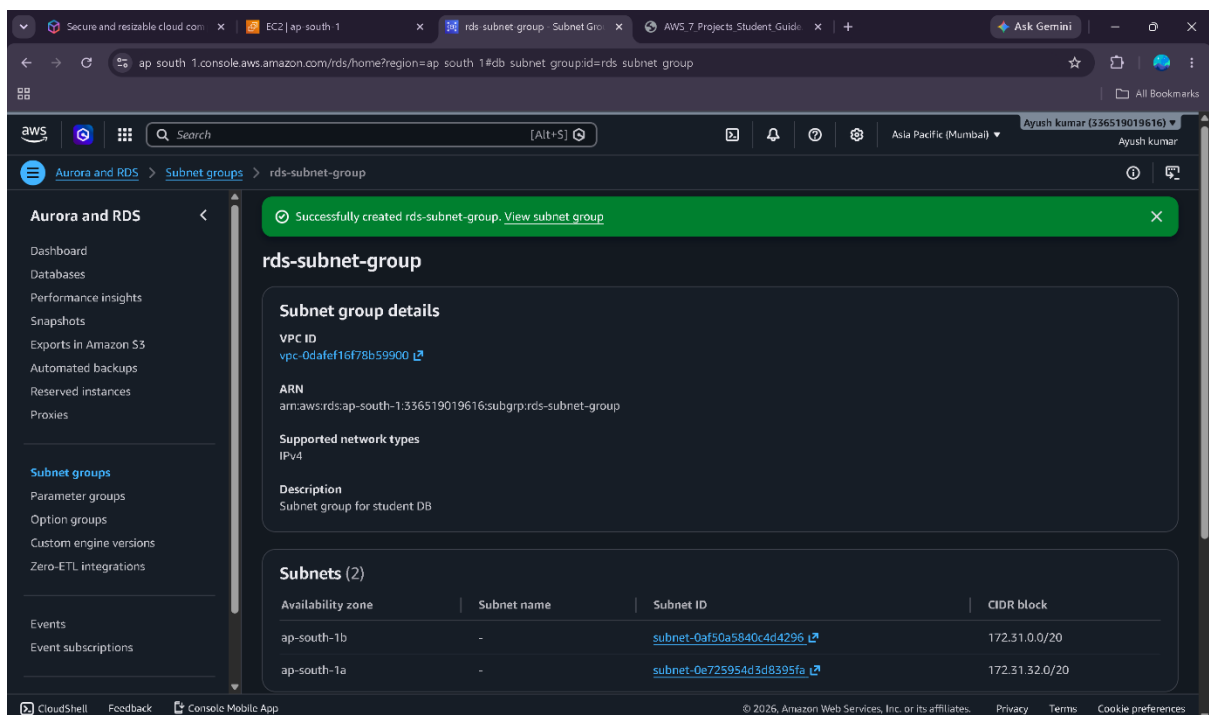
- **Inbound Rule:** MySQL/Aurora (Port 3306)
- **Source:** The Security Group ID of the existing EC2 instance.



Step 2: Creating an RDS Subnet Group

To deploy an RDS instance, a DB Subnet Group is required. This group defines which subnets and Availability Zones (AZs) the database can use.

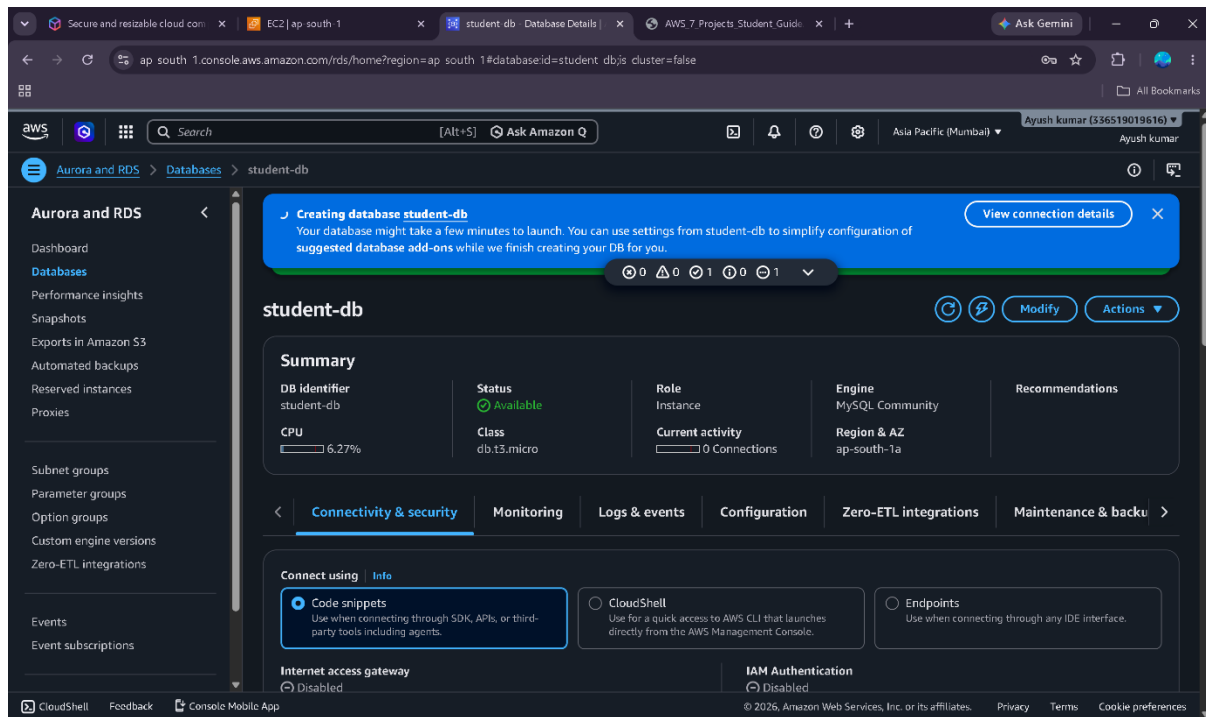
- **Action:** Navigated to RDS Console -> Subnet Groups -> Create DB Subnet Group.
- **Configuration:** Selected at least two subnets in different Availability Zones for high availability requirements.



Step 3: Launching the RDS Instance

The MySQL database was launched with the following configurations:

- **Engine:** MySQL
- **Template:** Free Tier (to avoid costs).
- **DB Instance Identifier:** student-db
- **Credentials:** Configured a Master Username and a secure Password.



Step 4. Database Connection & Verification

The connection to the remote RDS endpoint was successfully authenticated from the EC2 terminal using the primary endpoint routing:

```
mysql -h student-db.c92smm6g2drd.ap-south-1.rds.amazonaws.com -u admin -p
```

Database Operations (SQL Script Executed):

Upon establishing a successful database session, the following schema and data mutations were executed seamlessly:

```
CREATE DATABASE studentdb; USE studentdb;
CREATE TABLE students ( id INT PRIMARY KEY, name VARCHAR(50) );
INSERT INTO students VALUES (1, 'Satya');
SELECT * FROM students;
```

Final Output & Results: The database returned the following structured query result, confirming full end-to-end connectivity, proper IAM/Security Group routing, and successful data validation:

id | name 1 | Satya

```
ec2-user@ip-172-31-36-219:~$ mysql -h student-db.c92smm6g2drd.ap-south-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 38
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> CREATE DATABASE studentdb;
MySQL [(none)]> USE studentdb;
Database changed
MySQL [studentdb]> CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50));
Query OK, 0 rows affected (0.047 sec)
MySQL [studentdb]> INSERT INTO students VALUES (1, 'Satya');
Query OK, 1 row affected (0.009 sec)
MySQL [studentdb]> SELECT * FROM students;
+----+-----+
| id | name |
+----+-----+
|  1 | Satya |
+----+-----+
1 row in set (0.001 sec)

MySQL [studentdb]>
```

4. Conclusion

This project successfully demonstrated the deployment of a managed relational database on AWS. By using RDS, we offload administrative tasks like hardware provisioning and software patching to AWS while maintaining a secure connection through refined Security Group rules.